SkillSwap

Marketplace de talento, mentorías y proyectos

Informe de decisiones de diseño

Problema · alternativa elegida · consecuencias · evidencia en código y pruebas

Trabajo Práctico Obligatorio — Procesos de Desarrollo de Software
Universidad Argentina de la Empresa (UADE)

Grupo 12
Sánchez Accinelli, Antonio
Repetto, Germán
Gandara, Alexis
García, Matías
Toiw, Lautaro

Docente: Matías Stricagnoli
Octubre de 2026

1. Introducción y alcance

SkillSwap es el backend de una plataforma donde profesionales ofrecen mentorías o servicios breves, las organizaciones publican desafíos y los usuarios contratan con créditos ficticios llamados **SkillCoins**. Un mismo usuario puede actuar como cliente, como proveedor o como ambos. La consigna pide una solución en Java con Hibernate/JPA y MySQL, sin Spring Boot, con foco en dominio, casos de uso, arquitectura, persistencia y pruebas; no se evalúa el diseño visual.

Este informe no pretende repetir el código, sino explicar *por qué* está construido como está. Para cada decisión relevante se describe el problema que había que resolver, la alternativa que elegimos (y a veces las que descartamos), las consecuencias que eso trae —buenas y malas— y dónde se puede comprobar en el código o en las pruebas. Esa es, justamente, la estructura que pide la consigna de entrega.

1.1 Qué cubre la solución

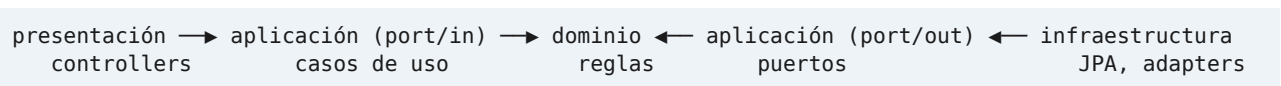
- **Perfiles y reputación**: registro con un *userId* de un servicio de identidad externo, datos editables, perfil público y reputación recalculada a partir de calificaciones verificadas (no editable a mano).
- **Publicaciones** de tres tipos (mentoría, servicio, desafío) con cinco estados y transiciones validadas por el dominio.
- **Búsqueda y recomendación**: filtros combinables, cinco criterios de orden, paginación y dos estrategias de recomendación intercambiables.
- **Contratación**: solicitud, aceptación con reserva y cupo transaccional, pago con SkillCoins, reprogramación acordada, cancelación con reintegro según política, finalización y calificación.
- **Notificaciones** por email, push o internas, con proveedores simulados detrás de interfaces reemplazables.
- **Persistencia** con Hibernate/JPA sobre MySQL, migraciones Flyway, control optimista de concurrencia y consultas paginadas sin N+1.

1.2 Fuera de alcance

Siguiendo la sección 16 de la consigna, no se implementa autenticación real, dinero real, chat, videollamadas, despliegue productivo ni diseño visual. El login es simulado: un servicio de identidad *mock* entrega el userId activo.

2. Arquitectura

La solución respeta la regla de dependencias de Clean Architecture: el código fuente solo apunta hacia adentro, hacia el dominio. El dominio no conoce a nadie; la aplicación conoce al dominio; la infraestructura y la presentación conocen a la aplicación. Las dependencias que apuntan hacia afuera (por ejemplo, necesitar guardar algo en una base) se resuelven con **puertos**: interfaces definidas en el lado interno e implementadas en el externo.



Capa	Qué contiene	Qué no debe conocer
Dominio	Entidades (Perfil, Publicacion, Franja, Contratacion, Wallet, MovimientoCredito, Calificacion, Notificacion, Promocion), value objects (SkillCoins, Puntaje, Habilidad, RangoPrecio, BloqueDisponibilidad), estados, eventos y observers, historial inmutable.	Interfaz gráfica, JPA, MySQL, SDKs.

Capa	Qué contiene	Qué no debe conocer
Aplicación	Nueve casos de uso (interfaces *port/in*), commands y DTOs, servicios que los implementan, puertos de salida y repositorios (IdentidadPort, TransaccionPort, RelojPort, CanalNotificacionPort, CanalNotificacionFactory, interfaces de repositorio), estrategias (políticas de cancelación y recomendación), decorators de precio, NotificadorObserver, jerarquía de errores con códigos.	Controllers concretos, SDKs externos, JPA.
Infraestructura	Entidades JPA y mappers, siete repositorios, adaptador transaccional, Flyway, adaptadores de identidad / email / push / interna, factory de canales, reloj del sistema.	Decisiones de negocio nuevas.
Presentación	Ocho interfaces de controller y sus implementaciones delgadas, el tipo Resultado, el traductor de errores y un menú de consola.	Persistencia y reglas de negocio.

2.1 Un caso de uso de punta a punta: aceptar una solicitud

El recorrido ilustra cómo colaboran las capas. La interfaz llama a *IContratacionController.aceptar(id)*. El controller (presentación) pide el perfil del usuario actual a *SesionUseCase*, que lo obtiene del puerto de identidad, y delega en *ContratacionUseCase.aceptar(actorId, id)*. El servicio (aplicación) abre una transacción mediante *TransaccionPort*, carga la contratación y la publicación por sus repositorios, verifica que el actor sea el proveedor, y le pide al **dominio** que haga el trabajo: *contratacion.aceptar()* valida la transición y *publicacion.reservarFranja()* valida franja libre y cupo. Se guardan ambos agregados, se confirma la transacción y, recién entonces, se despachan las notificaciones. Si algo falla en cualquier punto, la excepción sube hasta el controller, que la convierte en un *Resultado* con código de error, sin que la interfaz vea jamás una traza.

Lo importante es dónde vive cada decisión: **qué** se puede hacer lo decide el dominio; **en qué orden y dentro de qué transacción** lo decide la aplicación; **cómo se guarda** lo decide la infraestructura; y **cómo se muestra** lo decide la presentación. Ninguna capa invade a la otra.

2.2 Composición

Como no se usa Spring, la composición de objetos se hace a mano en un único lugar (*bootstrap/Composicion*), que es el único código que conoce a todas las capas a la vez. Allí se instancian repositorios, adaptadores, servicios y controllers, y se inyectan por constructor. Esto mantiene explícitas las dependencias y facilita reemplazarlas en las pruebas: los tests de aplicación arman el mismo grafo con repositorios en memoria y relojes controlados.

3. Modelo de dominio

3.1 Agregados y referencias por identificador

Cada agregado protege sus invariantes y expone comportamiento, no solo datos. Perfil, Publicacion, Contratacion, Wallet, Calificacion, Notificacion y Promocion se referencian entre sí **por identificador** y no por referencia de objeto. Esto evita grafos enormes cargados por accidente, deja claros los límites transaccionales y es lo que permite que el dominio sea independiente de JPA.

Agregado	Responsabilidad e invariantes principales
Perfil	Datos públicos, canal de notificación preferido, reputación. La reputación solo cambia mediante *recalcularReputacion* a partir de calificaciones; no existe un setter. La baja es lógica.

Agregado	Responsabilidad e invariantes principales
Publicacion	Ciclo de vida de cinco estados, franjas y cupo. Publicar exige título, categoría, precio no negativo y al menos una franja. Solo se edita en BORRADOR. Una franja ocupada o un cupo agotado no se pueden reservar.
Contratacion	Máquina de siete estados, precio final fijado al solicitar, propuesta de reprogramación pendiente, historial de transiciones y eventos pendientes de notificar. El autor no puede solicitar su propia publicación.
Wallet	Saldo en SkillCoins que nunca es negativo. Cada débito o crédito genera un movimiento. Un débito sin saldo falla sin modificar nada.
Calificacion	Solo participantes de una contratación COMPLETADA, una vez por autor, nunca hacia uno mismo.
Promocion	Descuento porcentual o de monto fijo con vigencia; las promociones vigentes se acumulan.

3.2 Value objects

SkillCoins encapsula el dinero ficticio con escala fija de dos decimales, aritmética propia y la operación *noNegativo*, que es la base de la regla «ninguna promoción puede producir un total negativo». **Puntaje** valida el rango 1 a 5 en construcción, **Habilidad** normaliza el texto para que «Java» y «java» sean la misma habilidad, y **RangoPrecio** y **BloqueDisponibilidad** validan su coherencia interna. Son inmutables y se comparan por valor.

3.3 Historial inmutable

Toda transición de estado, tanto de publicaciones como de contrataciones, registra fecha, actor y motivo en un *HistorialEstado* que solo admite alta: no tiene setters ni operaciones de modificación, y la persistencia únicamente inserta las entradas nuevas. Los estados de destino que representan una salida irregular (rechazo, cancelación) exigen un motivo no vacío.

4. Decisiones de diseño

A continuación se documentan las decisiones más importantes siguiendo el formato pedido: problema, alternativa elegida, consecuencias y evidencia.

D1. Escrow: el dinero queda retenido en la contratación

Problema	El pago, la prestación del servicio y el cobro del proveedor ocurren en momentos distintos. Había que decidir dónde está el dinero entre el pago y la finalización, y qué pasa si se cancela en el medio.
Alternativa elegida	Modelo de *escrow*. Al pagar se debita al cliente y el monto queda retenido en la propia contratación; al completar se acredita al proveedor. Si se cancela una contratación pagada, el cliente recibe el reintegro que dicta la política y el resto se acredita al proveedor como compensación. Si cancela el proveedor, el cliente recupera el 100 %.
Consecuencias	(+) Nadie cobra por un servicio no prestado y el reintegro es verificable: reintegro más compensación siempre suma el precio. (+) No hace falta una billetera de plataforma. (−) El dinero retenido no es consultable como saldo de nadie; se ve en la contratación. (−) Alternativa descartada: acreditar al proveedor al pagar, más simple pero obliga a «cobrarle de vuelta» en las cancelaciones.
Evidencia	*ContratacionService* (pagar, completar, cancelar); *ContratacionCancelacionServiceTest* verifica los montos exactos en los límites de cada política; casos de aceptación 3, 5 y 6.

D2. La reserva vive dentro de la contratación

Problema	La consigna habla de solicitudes, reservas y contrataciones. Modelarlas como tres entidades duplicaría estados y complicaría las transacciones.
Alternativa elegida	Una única entidad *Contratacion* cuya máquina de estados cubre desde la solicitud (PENDIENTE) hasta el cierre. La reserva es el vínculo entre la contratación y una *Franja* de la publicación, que se ocupa al aceptar y se libera al cancelar.
Consecuencias	(+) Un solo agregado con un solo historial; las operaciones de aceptar, reprogramar y cancelar tocan exactamente dos agregados (contratación y publicación) en una transacción. (−) No existe una tabla «reserva» propia; si la cátedra la esperara, se extrae sin cambiar el contrato de los casos de uso.
Evidencia	*Contratacion*, *Publicacion.reservarFranja / liberarFranja / reprogramarReserva*; *ContratacionReprogramacionServiceTest*.

D3. Dominio puro y entidades JPA separadas

Problema	Si las entidades de dominio llevaran anotaciones JPA, el dominio dependería de la tecnología de persistencia, violando la regla de dependencias que exige la consigna.
Alternativa elegida	El dominio son clases Java simples (POJOs). En infraestructura existen entidades **JpaEntity* y *mappers* que traducen en ambos sentidos. Para reconstruir un agregado desde la base se usan métodos *reconstruir(...)* del propio dominio, que restauran el estado sin disparar reglas de negocio ni eventos.
Consecuencias	(+) El dominio se prueba sin base de datos ni contenedor y puede cambiar de motor sin tocarse. (−) Más código: una entidad y un mapper por agregado. (−) Hay que cuidar que el mapper no pierda información; para eso hay pruebas de ida y vuelta por repositorio.
Evidencia	Paquetes *domain.model* (sin imports de *jakarta.persistence*) e *infrastructure.persistence.jpa*; pruebas *Jpa*RepositoryTest*.

D4. Una transacción por caso de uso; notificaciones después del commit

Problema	Aceptar, pagar y cancelar modifican varios agregados y movimientos de dinero. Un fallo a mitad de camino no puede dejar estados a medias (por ejemplo, dinero debitado sin pago registrado). Por otro lado, no se debe avisar a nadie de algo que luego se revierte.
Alternativa elegida	Cada operación que modifica estado corre dentro de *TransaccionPort.ejecutar(...)*, implementado en infraestructura por un adaptador reentrante sobre un *EntityManager* por unidad de trabajo. Los eventos de dominio se acumulan en la contratación y el observador los despacha recién **después** del commit, en su propia transacción.
Consecuencias	(+) Atomicidad: débito, crédito y cambio de estado se confirman o se revierten juntos. (+) Nunca se notifica un hecho que no ocurrió. (−) Si el envío de una notificación falla, la operación ya confirmada no se deshace; se registra el error y no se pierde el hecho de negocio. Es un compromiso deliberado.
Evidencia	*JpaTransaccionAdapter*; servicios en *application.service*; *NotificadorObserverTest* verifica que un canal que falla no deshace la operación.

D5. Control optimista contra la sobreventa y el doble cobro

Problema	Dos aceptaciones simultáneas podrían consumir el último cupo, y dos pagos simultáneos podrían debitar dos veces.
Alternativa elegida	Control optimista con *@Version* en Perfil, Publicacion, Contratacion y Wallet, más una verificación de versión previa al guardado. Los conflictos se traducen a una excepción de aplicación (*ConflictoConcurrenciaException*) con un código de error uniforme para que la interfaz pueda proponer reintentar. Además, el propio dominio rechaza reservar sobre una franja ocupada o con cupo agotado.
Consecuencias	(+) Sin bloqueos largos ni deadlocks; escala mejor que el bloqueo pesimista para este perfil de uso. (−) El perdedor de una carrera recibe un error y debe reintentar. (−) Al reprogramar solo cambian franjas, así que se incrementa una revisión de la publicación para que el control de versión lo detecte.
Evidencia	*@Version* en las entidades JPA; *ErroresJpa*; casos de aceptación 4 (versión en memoria y versión concurrente sobre H2).

D6. Reputación derivada, no editable

Problema	La reputación no puede editarse manualmente: si fuera un campo asignable, cualquiera podría falsearla.
Alternativa elegida	El perfil no expone ningún setter de reputación. El único camino es *recalcularReputacion(calificaciones)*, que promedia las calificaciones verificadas. Se invoca en la misma transacción que registra una calificación nueva. Se guarda como columna para poder ordenar y filtrar por reputación sin recalcular en cada consulta.
Consecuencias	(+) Invariante garantizada por el diseño, no por convención. (+) Búsquedas por reputación eficientes. (−) Es un dato derivado persistido: si alguna vez se importaran calificaciones por fuera del caso de uso habría que recalcular.
Evidencia	*Perfil.recalcularReputacion*; *CalificacionService*; caso de aceptación 10.

D7. Política de cancelación por publicación, umbrales configurables

Problema	El reintegro debe ser 100 %, 50 % o 0 % según la anticipación configurable, y distintas publicaciones pueden querer reglas distintas.
Alternativa elegida	Cada publicación guarda un tipo de política (FLEXIBLE o ESTRICTA). Una fábrica construye la estrategia correspondiente con umbrales tomados de una configuración (por defecto: flexible 24 h / 2 h, estricta 72 h / 48 h). El servicio solo pregunta a la estrategia cuánto reintegrar.
Consecuencias	(+) Agregar una política nueva es agregar una clase y una línea en la fábrica. (+) Los umbrales no están dispersos en el código. (−) Los umbrales son globales; no se pueden personalizar por publicación sin persistirlos.
Evidencia	*PoliticaCancelacion*, *PoliticaFlexible*, *PoliticaEstricta*, *PoliticaCancelacionFactory*; *PoliticaCancelacionTest*; caso de aceptación 6.

D8. Errores uniformes hacia la interfaz

Problema	La interfaz no debe conocer excepciones del dominio ni trazas, y los errores de aplicación deben tratarse de forma uniforme (consigna, sección 9).
Alternativa elegida	Los controllers nunca lanzan: devuelven un *Resultado<T>* que es éxito con valor o error con un código (VALIDACION, NO_ENCONTRADO, NO_AUTORIZADO, REGLA_DE_NEGOCIO, TRANSICION_INVALIDA, SALDO_INSUFICIENTE, CUPO_AGOTADO, FRANJA_NO_DISPONIBLE, CONFLICTO_CONCURRENCIA, ERROR_INTERNO) y un mensaje legible. Un único traductor mapea excepciones de dominio y de aplicación a esos códigos; todo lo inesperado se convierte en un error genérico.
Consecuencias	(+) La interfaz decide su reacción por código, sin *try/catch*. (+) Los controllers quedan delgados. (−) Hay que mantener la tabla de traducción cuando aparece una excepción nueva.
Evidencia	*presentation.ManejadorErrores*, *presentation.Resultado*; controllers en *presentation.controller*.

D9. Organización como tipo de perfil

Problema	Las organizaciones publican desafíos y los profesionales mentorías o servicios, pero ambos contratan, pagan y califican de la misma forma.
Alternativa elegida	Un único agregado *Perfil* con un tipo (PROFESIONAL u ORGANIZACION). La regla «solo una organización publica desafíos y solo un profesional publica mentorías o servicios» vive en la creación de la publicación.
Consecuencias	(+) Todo el circuito de contratación, wallet, reputación y notificaciones se reutiliza sin duplicar. (−) Si las organizaciones tuvieran datos muy distintos (razón social, miembros) habría que separar el modelo.
Evidencia	*TipoPerfil*, *Publicacion.crearBorrador*; *PublicacionTest*.

D10. Esquema por migraciones, nunca por generación automática

Problema	Dejar que Hibernate cree el esquema oculta errores de modelado y es inaceptable en una entrega (la consigna lo prohíbe).
Alternativa elegida	Flyway crea y versiona el esquema (*V1__esquema.sql*) al arrancar; Hibernate se configura con *hbm2ddl=validate*, de modo que la aplicación no arranca si una entidad no coincide con las tablas. Las credenciales llegan por variables de entorno, no por archivos versionados.
Consecuencias	(+) El esquema es un artefacto revisable y reproducible; desfasajes entre código y tablas fallan temprano. (−) Hay que escribir y mantener el SQL a mano y mantenerlo portable entre MySQL y H2 para las pruebas.
Evidencia	*db/migration/V1__esquema.sql*, *BaseDeDatos*, *BaseDeDatosTest*.

D11. Búsqueda paginada sin N+1

Problema	Un listado que carga publicaciones y luego consulta autor, promociones y colecciones fila por fila dispara cientos de consultas. Además, combinar *JOIN FETCH* de colecciones con paginación obliga a Hibernate a paginar en memoria.
Alternativa elegida	La búsqueda ejecuta primero un *count*, luego una consulta de identificadores paginados y por último una consulta de carga con *JOIN FETCH* por colección. Autores y promociones se cargan **por lote** con una consulta cada uno. Se configura *fail_on_pagination_over_collection_fetch* para que reintroducir el error falle fuerte. Los parámetros son siempre nombrados y los comodines de texto se escapan.
Consecuencias	(+) Cantidad de consultas acotada e independiente del tamaño de la página; sin inyección. (−) Tres consultas en vez de una; es el precio de paginar correctamente.
Evidencia	*JpaPublicacionRepository*, *ArmadorResumenes*, *CalculadorPrecios*; *BusquedaServiceTest* verifica una sola consulta de autores y de promociones por página.

5. Patrones de diseño

La consigna pide al menos cuatro patrones que resuelvan una variación real del problema, no una cantidad por sí misma. Aplicamos siete y dejamos dos afuera de forma deliberada. En cada caso se indica la variación concreta que justifica el patrón y la prueba que lo respalda.

Patrón	Variación que resuelve	Dónde está	Evidencia
State	El comportamiento y las transiciones válidas de una publicación (5 estados) y de una contratación (7 estados) cambian con el estado. Los *if* por estado se vuelven inmanejables.	*publicacion.estado.**, *contratacion.estado.**; base común que lanza *TransicionInvalidaException*; fábrica *desdeNombre*.	*PublicacionTest*, *FranjaYEstadosTest*, *ContratacionTest*; casos 2, 3 y 5.
Strategy	El reintegro por cancelación y la recomendación son algoritmos intercambiables que deben poder sumar variantes sin tocar a quien los consume.	*PoliticaCancelacion* (Flexible, Estricta); *EstrategiaRecomendacion* (Afinidad, Popularidad).	*PoliticaCancelacionTest*, *RecomendacionTest*; casos 6 y 7.
Decorator	Las promociones son acumulables y el número de combinaciones posibles crece; una jerarquía de subclases por combinación sería imposible.	*PrecioPublicacion* → *PrecioBase* → *PromocionPorcentaje* / *PromocionMontoFijo* sobre *PromocionDecorator*.	*PrecioPublicacionTest*; caso 9.
Observer	Un cambio en la contratación debe avisar a varios interesados sin que el dominio conozca email, push ni bandeja.	*Contratacion* (Subject), *ContratacionObserver*, *ContratacionEvent*; *NotificadorObserver* en aplicación.	*ContratacionObserverTest*, *NotificadorObserverTest*; caso 8.
Adapter	Identidad, email y push son proveedores externos con APIs propias que no controlamos y que deben ser reemplazables.	*IdentidadMockAdapter* sobre *ServicioIdentidadExterno*; *Email/Push/InternaCanalAdapter* sobre *EmailSdkMock* / *PushSdkMock*.	*IdentidadMockAdapterTest*, *CanalesNotificacionTest*; caso 8.
Simple Factory	Hay que elegir el canal, la política, la estrategia o el estado a partir de un valor (preferencia del usuario, texto persistido).	*CanalNotificacionFactoryImpl*, *PoliticaCancelacionFactory*, *EstrategiaRecomendacionFactory*, *PrecioPublicacionFactory*, *EstadoXFactory*.	*CanalesNotificacionTest*, *PoliticaCancelacionTest*.

Patrón	Variación que resuelve	Dónde está	Evidencia
Facade	El perfil público junta datos, reputación, calificaciones recientes y publicaciones activas de varias fuentes; la interfaz no debería orquestarlo.	*PerfilService.verPerfilPublico* → *PerfilPublicoDto*.	*PerfilServiceTest*.

5.1 Patrones descartados y por qué

- **Composite**: sería razonable para un catálogo jerárquico de categorías y habilidades, pero el alcance no exige jerarquía; las categorías son etiquetas planas y agregar el patrón sería complejidad sin variación que lo justifique. Queda como evolución natural si se incorporan subcategorías.
- **Singleton**: no hay un recurso que requiera una única instancia global. La composición explícita por constructor evita el estado global oculto y simplifica las pruebas; el contexto de persistencia es único porque el compositor así lo decide, no porque el lenguaje lo imponga.

5.2 Cómo se combinan

Los patrones no están aislados. Un mismo evento recorre varios: la contratación cambia de estado (**State**), publica un evento (**Observer**), el observador consulta la preferencia del destinatario, la fábrica entrega el canal (**Simple Factory**) y el adaptador traduce la llamada al SDK del proveedor (**Adapter**). Cada uno resuelve un problema distinto y ninguno conoce los detalles del siguiente.

6. SOLID y GRASP

Principio	Cómo se evidencia
SRP	Cada clase tiene una razón de cambio: los estados deciden transiciones, los servicios orquestan, los mappers traducen, los controllers adaptan. *CalculadorPrecios* y *ArmadorResumenes* concentran una sola tarea auxiliar.
OCP	Se agrega un canal, una política, una estrategia o una promoción creando una clase y registrándola en su fábrica; no se modifican los consumidores.
LSP	Todas las políticas, estrategias, estados y canales son intercambiables detrás de su interfaz sin que el consumidor distinga cuál recibe.
ISP	Los casos de uso se exponen como nueve interfaces pequeñas y cohesivas en lugar de un servicio gigante; los controllers dependen solo de lo que usan.
DIP	El dominio y la aplicación dependen de abstracciones (repositorios, puertos); las implementaciones concretas viven afuera y se inyectan en el compositor.
Information Expert	Quien tiene los datos decide: *Wallet* decide si hay saldo, *Publicacion* si hay cupo, *Calificacion.registrar* quién puede calificar.
Creator	Los agregados se crean mediante métodos de fábrica con nombre de negocio (*solicitar*, *crearBorrador*, *abrir*) que validan al nacer.
Controller	Los controllers de presentación solo traducen entre la interfaz y los casos de uso; no contienen reglas ni acceden a persistencia.
Low coupling / High cohesion	Los paquetes se acoplan solo por interfaces; los agregados se referencian por id.
Polimorfismo	Reemplaza a los condicionales por estado, política, canal y estrategia.
Pure Fabrication / Indirection	Los puertos, los mappers y las fábricas son clases sin analogía en el negocio que existen para mantener el desacople.

Principio	Cómo se evidencia
Protected Variations	Las variaciones previsibles (proveedores externos, motor de base de datos, políticas) quedan detrás de interfaces estables.

7. Persistencia

7.1 Modelo relacional

El esquema contiene tablas para perfiles (con sus habilidades, intereses y bloques de disponibilidad), publicaciones (con etiquetas, habilidades, franjas e historial), contrataciones (con historial y propuesta de reprogramación), wallets y movimientos de crédito, calificaciones, notificaciones y promociones. Las relaciones entre agregados son claves foráneas a identificadores; las colecciones propias de un agregado se modelan como tablas hijas.

Restricción	Qué garantiza
perfil.user_id único	Un solo perfil por usuario del servicio de identidad.
calificacion(contratacion_id, autor_id) único	Cada parte califica una sola vez cada contratación.
Índice *publicacion(estado, categoria, precio_base)*	Búsquedas por estado, categoría y precio.
Índices de historial y de *notificacion(destinatario_id, leida)*	Consultas de trazabilidad y bandeja de entrada.
Columna *version*	Control optimista de concurrencia en los agregados que lo necesitan.

7.2 Mapeo del estado

El estado se persiste como texto y el objeto de estado se reconstruye mediante la fábrica *desdeNombre*, la misma idea que vimos en clase con *EstadoPedido* (texto persistido más objeto transitorio reconstruido al cargar). La diferencia es que acá la reconstrucción la hace el propio método *reconstruir* del dominio, de modo que las entidades JPA no necesitan callbacks de ciclo de vida.

7.3 Límites transaccionales

Operación	Qué se confirma o revierte junto
Aceptar	Estado de la contratación + reserva de franja + cupo de la publicación.
Pagar	Débito al cliente + movimiento + estado de la contratación.
Completar	Estado de la contratación + acreditación al proveedor + movimiento.
Cancelar	Estado + liberación de franja y cupo + reintegro al cliente + compensación al proveedor.
Calificar	Calificación + recálculo de la reputación del destinatario.
Registrar perfil	Perfil + wallet con bono inicial.

7.4 Datos de demostración

La demo siembra, de forma idempotente y a través de los propios casos de uso, tres profesionales y una organización con habilidades, publicaciones publicadas con franjas relativas a la fecha actual y una promoción. Se hace por código y no por un script SQL porque las franjas deben ser futuras respecto del momento de ejecución.

8. Estrategia de pruebas

Las pruebas siguen la pirámide: muchas pruebas rápidas sobre reglas, menos sobre casos de uso con puertos simulados y unas pocas de integración. No se usaron librerías de *mocking*; las dependencias se reemplazan por **fakes** escritos a mano (repositorios en memoria, reloj controlable, identidad simulada), que hacen los tests más legibles y ejercitan comportamiento real.

Nivel	Qué prueba	Ejemplos
Dominio	Invariantes, transiciones válidas e inválidas, value objects, estrategias, decorators, observer, wallet.	*PublicacionTest*, *ContratacionTest*, *WalletTest*, *CalificacionTest*, *PoliticaCancelacionTest*, *PrecioPublicacionTest*, *RecomendacionTest*
Aplicación	Cada servicio con repositorios en memoria: permisos, recursos inexistentes, montos exactos, notificaciones.	*ContratacionServiceTest*, *ContratacionCancelacionServiceTest*, *BusquedaServiceTest*, *CalificacionServiceTest*
Infraestructura	Adaptadores de identidad y canales; repositorios JPA contra una base H2 en modo MySQL.	*IdentidadMockAdapterTest*, *CanalesNotificacionTest*, *Jpa*RepositoryTest*
Aceptación	Los diez casos obligatorios de la consigna, en versión en memoria y en versión sobre JPA/H2 a través de los controllers.	*Caso01…Caso10*

8.1 Casos negativos exigidos

La consigna pide cubrir explícitamente transición inválida, saldo insuficiente, cupo agotado, doble pago y calificación no autorizada. Todos tienen pruebas dedicadas. En particular, para el saldo insuficiente se verifica no solo el error, sino que la contratación siga en su estado y que saldos y movimientos queden intactos.

8.2 Los diez casos de aceptación

#	Caso (sección 15)	Qué demuestra
1	Publicar una mentoría válida y encontrarla con búsqueda filtrada	Alta, publicación y búsqueda combinada.
2	Rechazar la publicación de una oferta incompleta	State y validación de completitud.
3	Solicitar, aceptar y pagar con saldo suficiente	Flujo principal y escrow.
4	Evitar que dos aceptaciones consuman el último cupo	Cupo, franja y control optimista.
5	Rechazar un pago con saldo insuficiente sin modificar nada	Atomicidad y wallet nunca negativo.
6	Cancelar con dos políticas diferentes	Strategy: misma anticipación, distinto reintegro.
7	Cambiar la estrategia de recomendación sin tocar el consumidor	Strategy intercambiable.
8	Dos tipos de notificación ante un mismo evento	Observer, Adapter y Simple Factory.
9	Promociones acumulables sin precio negativo	Decorator y acotado en cero.
10	Completar, calificar una sola vez y recalcular reputación	Reglas de calificación y reputación derivada.

9. Alcance de esta entrega y trabajo futuro

Se prefiere ser explícito sobre los límites actuales, porque son decisiones conscientes y no omisiones.

- **Interfaz gráfica**: las interfaces de los controllers están pensadas como contrato de integración; la conexión con la interfaz provista por la cátedra es un paso posterior y probablemente requiera ajustar firmas.
- **Cancelación de publicaciones con contrataciones activas**: hoy no dispara un reintegro automático de las contrataciones en curso. La evolución natural es impedirlo o cancelarlas en cascada con reintegro total.
- **Retiro de una solicitud pendiente por el cliente**: el dominio solo permite al proveedor rechazarla.
- **Composite de categorías y comisiones de plataforma**: quedaron fuera por no ser parte del alcance mínimo; el diseño (Decorator de precio, categorías como etiquetas) permite incorporarlos sin romper lo existente.
- **Cobertura**: la medición con JaCoCo contra la meta sugerida del 80 % en dominio y aplicación se realiza al ejecutar la construcción con Maven.
- **Ejecución del conjunto de pruebas sobre MySQL/H2**: las pruebas de persistencia y de aceptación sobre JPA se ejecutan con la construcción de Maven en el entorno del equipo.

10. Cómo ejecutar

```
mvn verify                                  # compila, corre las pruebas y genera cobertura
mvn exec:java -Dexec.args="--h2"             # demo completa sobre H2 en memoria
mvn exec:java -Dexec.args="--h2 --menu"      # demo y menú de consola
mvn exec:java -Dexec.args="--mysql"          # contra MySQL (SKILLSWAP_DB_URL / _USER / _PASSWORD)
```

La demo recorre publicar → solicitar → aceptar → pagar → iniciar → completar → calificar, e incluye una cancelación con reintegro parcial y un pago rechazado por saldo insuficiente, mostrando en cada paso el actor, el estado y los saldos. El README del repositorio detalla requisitos, variables de entorno y la matriz de cumplimiento de la consigna.